

Rules and regulations that apply on-site: Some industries have strict rules or regulations that stop them from using public cloud services for sensitive workloads. In these situations, Knative lets teams build modern, scalable applications that meet security and compliance standards on their own infrastructure without having to worry about servers.

Control over networking in small steps: Another good thing is that it can work with advanced networking tools like Istio or Kourier. These technologies give you a lot of control over routing, security policies, and how well you can see what's going on. As a result, platform teams can set up advanced networking settings that many managed serverless platforms can't do.

When companies use Knative in production, they often build systems that strike a balance between scalability, security, and operational efficiency. Knative runs on top of Kubernetes, so a lot of best practices come from how to set up a Kubernetes cluster, but they also consider things that are unique to serverless.

Design for a multi-tenant cluster

In a common production scenario, multiple teams or applications share the same Kubernetes infrastructure in a multi-tenant cluster. In such places, it's important to keep things in control.

Separation of namespaces: Namespaces help keep things organised in a Kubernetes cluster. Each team or project can work in its own namespace, which lets them deploy Knative services, configurations, and event resources separately. This separation lowers the chance of team conflicts and makes it easier to manage access.

RBAC limits: Another important way to manage multi-tenant environments is through role-based access control (RBAC). Kubernetes RBAC policies say who can create, change, or delete resources.

Networking and ingress

Setting up the gateway: An ingress layer is what Knative uses to make services available to the outside world. A lot of deployments work with a service mesh like Istio or use lightweight ingress controllers like Kourier.

Services inside and outside the company: Not every service needs to be public in a lot of architectures. Some services only work inside the cluster, like internal APIs or background processors. Knative lets teams set up services so that they can only be accessed from inside the cluster or from the outside.

Autoscaling in the real world

One of the most important parts of Knative is autoscaling, but in real-world deployments, it needs to be carefully tuned to get the best performance.

Tuning for concurrency: Knative lets administrators set the maximum number of requests that a single container instance can handle at the same time. Changing the limits on concurrency helps find a balance between latency and resource use. Lower concurrency can speed up response times, while higher concurrency can cut down on the number of instances that are running.

Preventing cold starts: Because Knative can scale services down to zero, the first request after a period of inactivity may have to wait longer to start. Teams often get around this by keeping a minimum number of warm instances or speeding up the time it takes for containers to start up.

Architectures based on events

One of Knative's best features is that it supports event-driven architectures, which let services respond to events instead of just waiting for requests to come in. This method makes systems more scalable, separates their parts, and lets data streams be processed in real time.

A model based on brokers and triggers is used by Knative Eventing.

In the system, a broker is like an event router. It gets events from producers and sends them to the right consumers. Triggers set the rules for filtering and routing that decide which events should go to which services. This architecture lets developers make flexible pipelines where services can respond to events without having to be directly connected to each other.

Observability and operations

To run serverless workloads in production, you need strong monitoring and observability tools. Because serverless services can change size and run for short periods of time, it's very important to be able to see how the system is working.

Logging that is organised: Structured logging is popular in cloud-native environments because it makes log entries that machines can read. Logs are structured data instead of plain text messages. This makes it easier to search, filter, and analyse them automatically. This method makes it easier to fix problems and helps teams find problems faster in distributed systems.

Working with Prometheus: Collecting metrics is another important part of observability. Knative works well with monitoring tools like Prometheus, which keeps track of things like request rates, latency, and resource usage. These metrics let operators keep an eye on when the application is working, spot problems, and set up automatic alerts when certain levels are reached.

Fixing problems in production

Even systems that are well-designed fail from time to time, which is why debugging tools and deployment strategies are so important. With Knative's revision-based deployment model, operators can quickly go back to an older version of a service if a new one causes problems. Because each revision can't be changed, rolling back means sending traffic back to a stable revision.